

- 1.[10] Consider the following scenario: An office is used by several workers sharing a single printer. As a simplification of a real life scenario, it is assumed that the printer is able to print any number of jobs, independently of the number of pages those jobs have, before running out of toner. The toner is replaced by a technician whenever necessary.
- a.[5] Provide an FSP description of the above scenario. You must provide also a brief description of the intended behavior for each one of the processes you define.
Hint: Possible processes for the above scenario are PRINTER, USER and TECHNICIAN.
- b.[5] Provide a Petri nets (any kind) description of the above scenario.

Q1.a

FSP:

const $N = 3$
range Jobs = $0 \dots N$

PRINTER = PRINTER[N],
PRINTER[j; Jobs] = (when $(j = 0)$ replace_toner $\rightarrow$ PRINTER[N]
| when $(j > 0)$ print $\rightarrow$ PRINTER[j-1]).

USER = (print $\rightarrow$ USER).
range Users = $1 \dots N$; (N is defined previously, const $N = 3$)

// USERS = (forall $[i: \text{Users}]$ user[i]: USER).

TECHNICIAN = (replace_toner $\rightarrow$ TECHNICIAN).

// OFFICE = (PRINTER || USERS || TECHNICIAN)

{ user[Users]. print / give_paper,

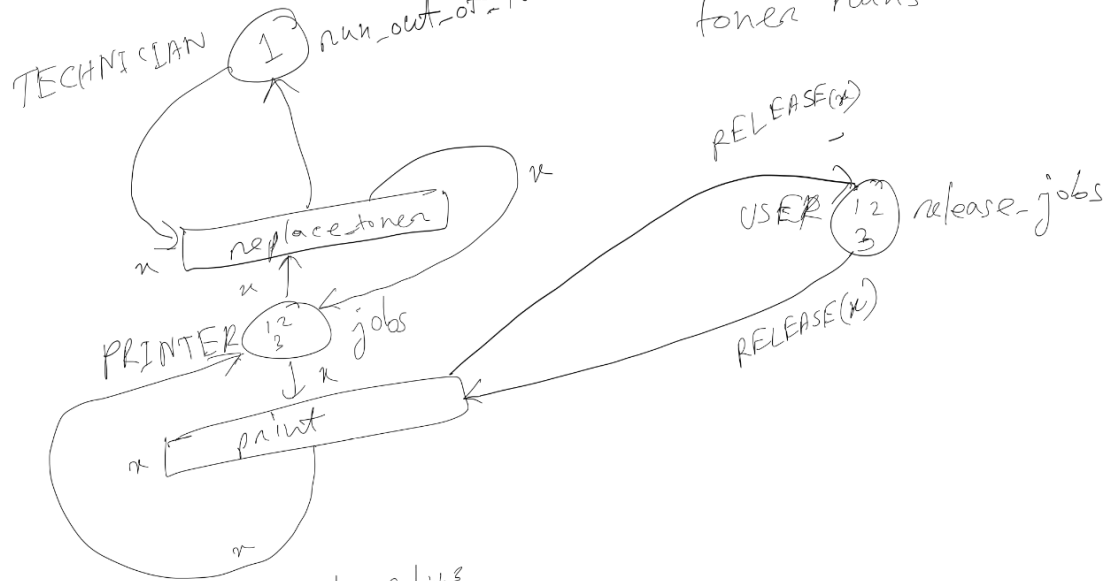
user[Users]. replace_toner / replace_toner }.

TECHNICIAN

You must

Description:

3 Users, 3 jobs → Assumption: Each user prints exactly once before toner runs out:



colour PRINTER = with job 1 | job 2 | job 3

colour USER = with u1 | u2 | u3

colour TECHNICIAN = with t1

RELEASE : USER → PRINTER

var x : PRINTER

fun RELEASE x = case of u1 ⇒ job 1 | u2 ⇒ job 2 | u3 ⇒ job 3

2.[15] Two warring neighbours are separated by a field with wild berries. They agree to permit each other to enter the field to pick berries, but also need to ensure that only one of them is ever in the field at a time. After negotiation, they agree to the following protocol. When a one neighbour wants to enter the field, he raises a flag. If he sees his neighbour's flag, he does not enter but lowers his flag and tries again. If he does not see his neighbour's flag, he enters the field and picks berries. He lowers his flag after leaving the field.

- a.[8] Model this algorithm for two neighbours $n1$ and $n2$. Specified the required *safety* properties for the field and check that it does indeed ensure mutually exclusive access. Specify the required *progress* properties for the neighbours such that they both get to pick berries given a fair scheduling strategy. Are any adverse circumstances in which neighbours would not make progress? What if the neighbours are greedy?

Hint: The following *FSP* can be used to model the flags.

```
const True = 1 const False = 0
range Bool = False..True
set BoolActions={setTrue,setFalse,[False],[True]}
BOOLVAR = VAL[False],
VAL[v:Bool] = ( setTrue -> VAL[True]
                | setFalse -> VAL[False]
                | [v] -> VAL[v] ).
```

- b.[7] Model this algorithm for two neighbours using Petri nets (any kind).

Q2.a

```
const False = 0
const True  = 1
range Bool  = False..True
set    BoolActions = {setTrue, setFalse, [False], [True]}

BOOLVAR = VAL[False],
VAL[v:Bool] = (setTrue  -> VAL[True]
               |setFalse -> VAL[False]
               |[v]      -> VAL[v]
               ).

||FLAGS = (flag1:BOOLVAR || flag2:BOOLVAR).

NEIGHBOUR1 = (flag1.setTrue -> TEST),
TEST       = (flag2[b:Bool] ->
              if(b) then
                (flag1.setFalse -> NEIGHBOUR1)
              else
                (enter -> exit -> flag1.setFalse -> NEIGHBOUR1)
              )+({flag1,flag2}.BoolActions).

NEIGHBOUR2 = (flag2.setTrue -> TEST),
TEST       = (flag1[b:Bool] ->
              if(b) then
                (flag2.setFalse -> NEIGHBOUR2)
              else
                (enter -> exit-> flag2.setFalse -> NEIGHBOUR2)
              )+({flag1,flag2}.BoolActions).

property SAFETY = (n1.enter -> n1.exit -> SAFETY | n2.enter -> n2.exit
-> SAFETY).

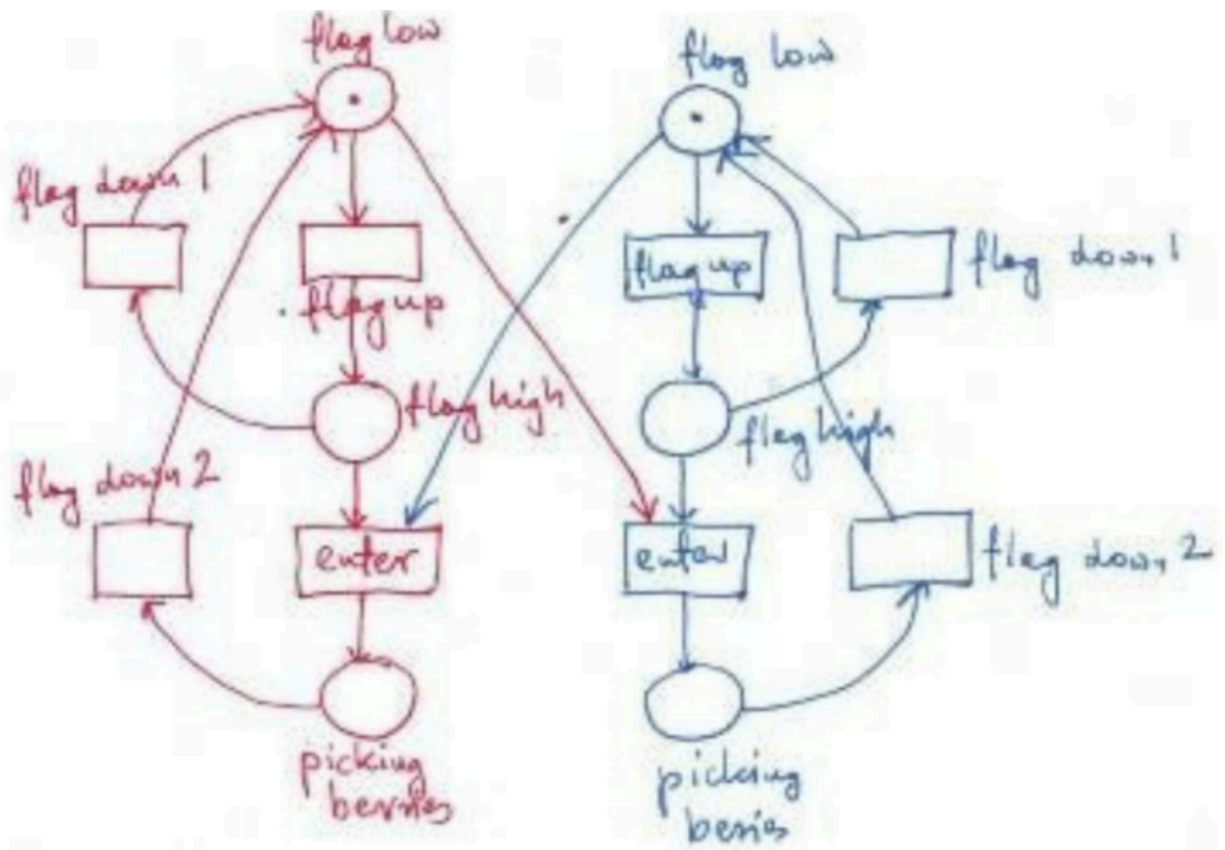
||FIELD = (n1:NEIGHBOUR1 || n2:NEIGHBOUR2 || {n1,n2}::FLAGS ||
SAFETY).

progress ENTER1 = {n1.enter} //NEIGHBOUR 1 always gets to enter
progress ENTER2 = {n2.enter} //NEIGHBOUR 2 always gets to enter

/* greedy neighbours - make setting the flags high priority -
eagerness to enter*/
||GREEDY = FIELD << {{n1,n2}.{flag1,flag2}.setTrue}.

/* progress violations show situation where neither neighbour enters
* each continually retests the lock
*/
```

Q2.b



3.[30] A cook puts burgers in a pot. A client checks if there is at least one burger in the pot, and if so, the client must take one.

(a)[20] Assuming two clients, this can be modelled as

```
range Burgers = 0..2
CLIENT = ( check -> get -> CLIENT ).

POT = POT[0],
POT[p: Burgers] = ( when p > 0 check -> POT[p]
  | get -> POT[p-1]
  | fill[n: Burgers] -> POT[n] ).

COOK = ( fill[p: 1..2] -> COOK )+{fill[0]}.
||DS = ( c1: CLIENT || c2: CLIENT || POT || COOK )
/{ {c1, c2}.check/check, {c1, c2}.get/get }.
```

(i)[12] Is there a trace belonging to process DS leading to an error state? If so, give the trace.

(ii)[8] We wish a client to obtain exclusive access to the pot. That is, two clients cannot check the pot at the same time, and when a client checks, he/she must take a burger (if there is at least one).
Show how to modify the given model such that this behaviour is ensured.

(b)[10] Provide a solution with any kind of Petri nets. Note that the cook cannot put a burger in a full pot.

Q3.a(i)

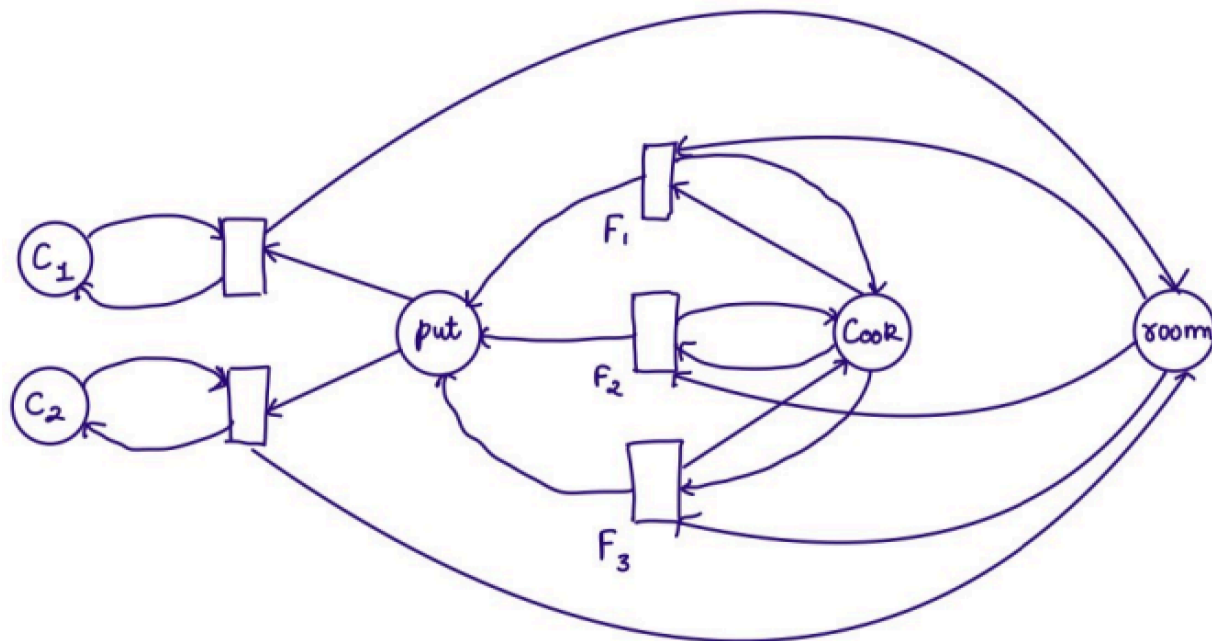
```
Trace to property violation in POT:
fill.1
c1.check
c2.check
c1.get
c2.get
```

In this trace, both clients check the pot and find that there are no burgers. Despite no burger being present in the pot, the clients will attempt to get a burger, this will lead to a error state.

Q3.a(ii)

```
POT = POT[0]
POT[p: Burgers] = (when (p>0) check
  ↪ -> get -> POT[p-1] |
  ↪ fill[n: Burgers] -> POT[n
  ↪ ]).
```

Q3.b



- 4.[20] The dining savages: A tribe of savages eats communal dinners from a large pot capable of holding M servings of stewed missionaries. When a savage wants to eat, he helps himself from the pot, unless it is empty, in which case he waits until the cook refills the pot. If the pot is empty, the cook refill the pot with M servings. The behavior of the savages and the cook is described by:

```
SAVAGE = ( get_serving -> SAVAGE ) .
COOK = ( fill_pot -> COOK ) .
```

- (a)[10] Model the behavior of the pot and of the system as FSP processes.
 (b)[10] Model the behaviour of the pot with Petri Nets (any kind, your choice)

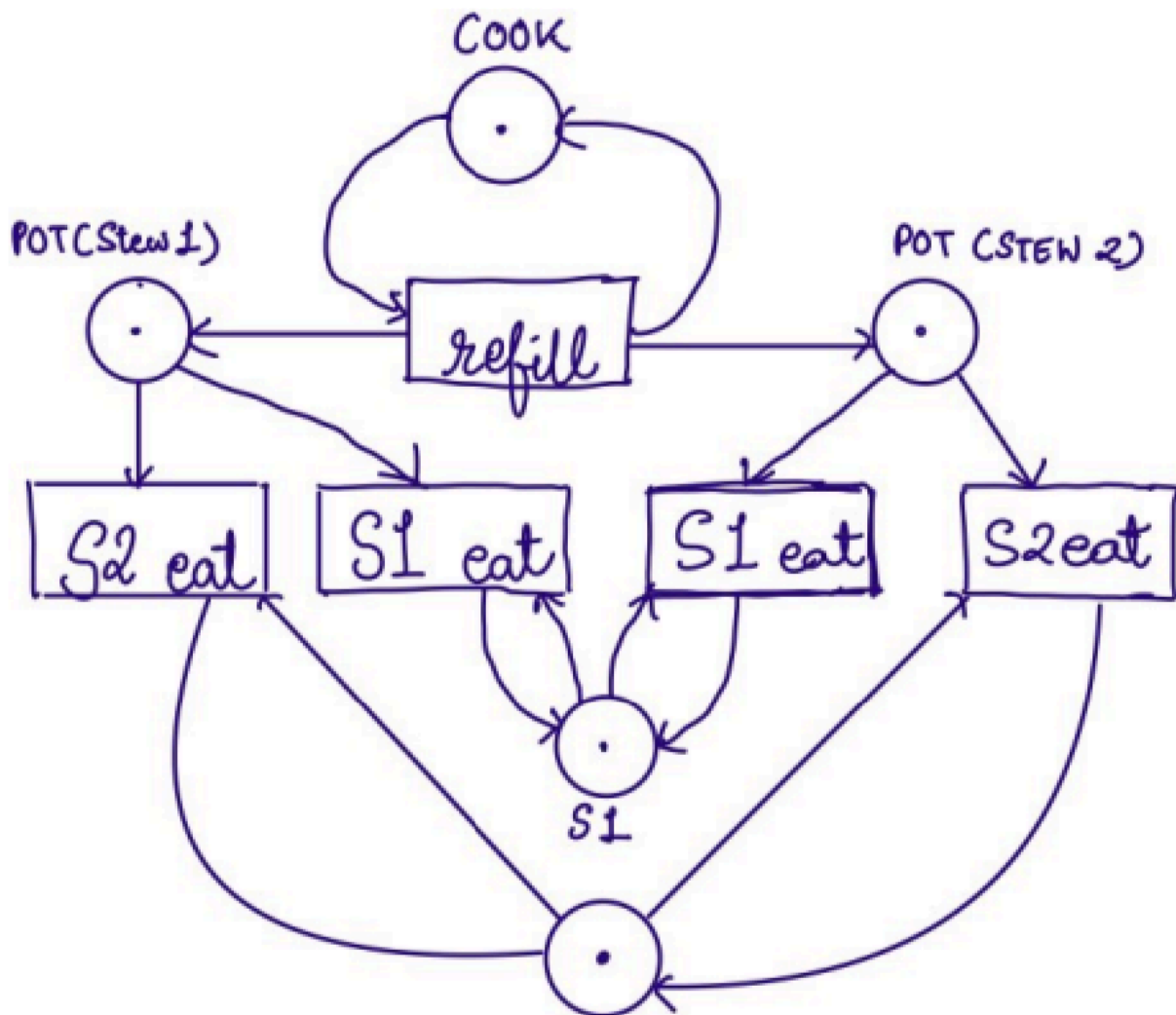
4.a

```

1  const M = 5
2
3  SAVAGE = (getsserving -> SAVAGE).
4  COOK = (fillpot -> COOK).
5
6  POT = SERVINGS[0],
7  SERVINGS[i:0..M] = (when (i == 0) fillpot -> SERVINGS[M]
8                      |when (i > 0) getsserving -> SERVINGS[i - 1])).
9
10 ||SAVAGES = (SAVAGE || COOK || POT).

```

4.b



- 5.[10] In an operating system, a binary semaphore is used to control access to the console. The console is used by user and system processes. Write down a model with FSP for this system. Discuss when user processes may be denied access to the console.

```
1 set Processes = {user[1..2], system[1..2]}
2
3 BSEMA = (up -> down -> BSEMA).
4 PROCESS = (console.up -> console.down -> PROCESS).
5
6 /* system processes have higher priority than user processes */
7 || OS = (Processes:PROCESS || Processes::console:BSEMA)>>{user}.
```

- 6.[9] Let

$$\begin{aligned} \text{property } \text{Alpha} &= (a \rightarrow b \rightarrow \text{Alpha} \mid b \rightarrow \text{Beta}) + \{d\} \\ \text{Beta} &= (c \rightarrow b \rightarrow \text{Alpha} \mid b \rightarrow \text{Beta}) \end{aligned}$$

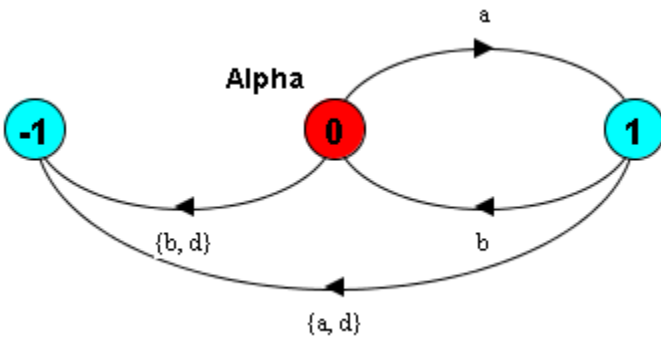
and

$$\begin{aligned} \text{Alpha} &= (a \rightarrow b \rightarrow \text{Alpha} \mid b \rightarrow \text{Beta}) + \{d\} \\ \text{Beta} &= (c \rightarrow b \rightarrow \text{Alpha} \mid b \rightarrow \text{Beta}) \end{aligned}$$

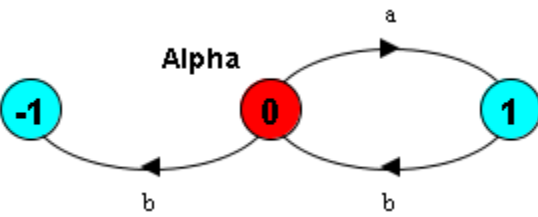
Let $\text{LTS}_{\text{propertyAlpha}}$ be the Labelled Transition System of the process `property Alpha`, and $\text{LTS}_{\text{Alpha}}$ be the Labelled Transition System of the process `Alpha`.

- a.[3] Produce $\text{LTS}_{\text{propertyAlpha}}$.
- b.[3] Produce $\text{LTS}_{\text{Alpha}}$.
- c.[3] Write a process `Alpha1` (but not property process) such that
$$\text{LTS}_{\text{propertyAlpha}} = \text{LTS}_{\text{Alpha1}}.$$

Q6.a



Q6.b



Q6.c

ALPHA1 = ALPHA_1.

ALPHA_1 = (a -> b -> ALPHA_1
 | b -> BETA_1
 | d -> STOP).

BETA_1 = (c -> b -> ALPHA_1
 | b -> BETA_1).

7.[10] *Simplified Multidimensional Semaphores* are defined as follows:

- The extended primitives *edown* and *eup* are atomic (indivisible) and each operates on a set of semaphore variables which must be initiated with non-negative integer value.
- *edown*($S_1, \dots, S_n$):
if for all $i, 1 \leq i \leq n, S_i > 0$ then for all $i, 1 \leq i \leq n, S_i := S_i - 1$
else block execution of calling processes
- *eup*($S_1, \dots, S_n$):
if processes blocked on ($S_1, \dots, S_n$) then awaken one of them
else for all $i, 1 \leq i \leq n, S_i := S_i + 1$

Model the *Simplified Multidimensional Semaphores* by FSPs for $n=2$ and maximal value of S_1, S_2 equal to 3.

Hint. *edown*(S_1, S_2) could be interpreted as just a synchronized execution of *down*(S_1) and *down*(S_2). Similarly for *eup*(S_1, S_2)

Q7

Generic semaphore is defined in FSP's as:

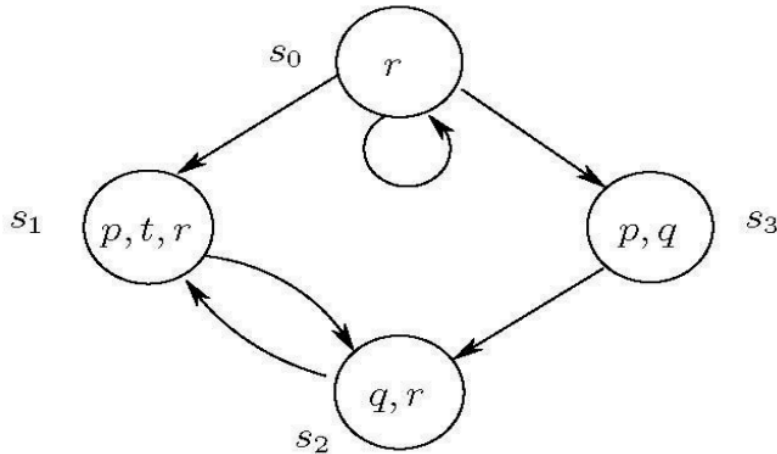
```
SEM(N=INITIAL_VALUE) = SEMA[N]
SEMA[v: Int] = (when(v<Max) up -> SEMA[v+1] |
               when(v>0) down -> SEMA[v-1])
```

Then a Simplified Multidimensional Semaphores over variables S_1, S_2 with the and maximal values of S_1, S_2 equal to 3, can be modelled by FSPs as follows:

```
SEMS1S2 (INITIAL1=3, INITIAL2=3) =
(S1:SEM(3) || S2:SEM(3))
/{S1.S2.up/S1.up, S1.S2.up/S2.up, S1.S2.down/S1.down,
 S1.S2.down/S2.down}
```

8.[18] This question deals with Model Checking.

(a)[6] Consider the system M defined below:



Determine whether $M, s_0 \models \varphi$ and $M, s_2 \models \varphi$ hold and justify your answer, where φ is the LTL or CTL formula:

(i)[3] $\text{EG } r$

(ii)[3] $\text{G}(r \vee q)$

(b)[3] Express in LTL:

*If the process is **enabled** infinitely often, then it **runs** infinitely often.*

(c)[3] Express in CTL:

*If the process is **enabled** infinitely often, then it **runs** infinitely often.*

(d)[6] Express in LTL and CTL:

A passenger entering the elevator at 5th floor and pushing 2nd floor button, will never reach 6th floor, unless 6th floor button is already lighted or somebody will push it, no matter if she/he entered an upwards or upward travelling elevator.

In this case you might use the following atomic predicates: $\text{floor}=2$, $\text{direction}=\text{up}$, $\text{direction}=\text{down}$, ButtonPres2 , $\text{floor}=6$, etc.

Q8

Similar to A3 Q7

(i) $EG \ \alpha$

$$\alpha \in L(s_0)$$

infinite path $s_0 \rightarrow s_0 \rightarrow s_0 \rightarrow \dots$

$$s_0, M, s_0 \models EG \ \alpha$$

$$\text{Next, } \alpha \in L(s_2) = \{\alpha, \alpha\}$$

$$s_0, M, s_2 \models EG \ \alpha; \text{ [future includes present]}$$

(a)
(ii) $G(n \vee q)$

$L(s_0) = \{n\}$
 $\therefore M, s_0 \models \varphi$
 $L(s_2) = \{q, n\}$
 $\therefore M, s_2 \models \varphi$

by inspection, all future states of s_0 and s_2 either have q or n or both.
 $s_0, M, s_1 \models \varphi$ and $M, s_2 \models \varphi$ hold.

... often, it n

(b) found from textbook:

<https://www.cas.mcmaster.ca/~se3bb4/course-files/LogicInComputerScience.pdf>

If the process is enabled infinitely often, then it runs infinitely often.
 $G F \text{ enabled} \rightarrow G F \text{ running}$.

(c) found from textbook. Ignore my handwritten solution.

The property that if the process is enabled infinitely often, then it runs infinitely often, is not expressible in CTL. In particular, it is not expressed by $AG AF \text{ enabled} \rightarrow AG AF \text{ running}$, or indeed any other insertion of A or E into the corresponding LTL formula. The CTL formula just given expresses that if every path has infinitely often enabled, then every path is infinitely often taken; this is much weaker than asserting that every path which has infinitely often enabled is infinitely often taken.

(b) If process is enabled infinitely often, it runs infinitely often.

LTL: $GF \text{ enabled} \rightarrow GF \text{ run}$

(c) LTL: $AG (GF \text{ enabled} \rightarrow GF \text{ run})$
 (d) if 6 is pressed, then elevator goes to 6th floor. and button to 6th floor not pressed from before
 If enter 5th floor and push button to 2nd floor, then elevator will never go to 6th floor.

LTL: $G((\text{floor} = 5 \wedge \text{ButtonPres}2) \Rightarrow (\neg \text{floor} = 6 \cup (\text{ButtonPres}6 \vee \neg \text{ButtonPres}6)))$

CTL: $AG((\text{floor} = 5 \wedge \text{ButtonPres}2) \Rightarrow A[\neg \text{floor} = 6 \cup (\text{ButtonPres}6 \vee \neg \text{ButtonPres}6)])$

9.[10] Consider the formulation of Smokers' Problem in plain English given in Lecture Notes 10, pages 5-7. The formulation of Dining Philosophers in the same style is in Lecture Notes 9 on page 7. Pages 16-18 of Lecture Notes 9, provide a solution to the Dining Philosophers problem with Elementary Petri Nets assuming that simultaneous picking forks (i.e. resources) is allowed, while pages 23-24 provide a solution to the same problem using Coloured Petri Nets, also assuming simultaneous picking forks (i.e. resources is allowed).

a.[5] Provide a solution to Smokers problem with Elementary Petri Nets and assuming that simultaneous picking (and delivery) resources is allowed (just mimic the solution for Dining Philosophers).

b.[5] Provide a solution to Smokers problem with Coloured Petri Nets and assuming that simultaneous picking (and delivery) resources is allowed (just mimic the solution for Dining Philosophers).

Q9(a) and Q9(b)

